PA#2 — Harness and quality gate · rubric

CSC13114 · project milestone, teams of up to 3 · 4% of the course · 100 points

The product is **LeeTKOB**: read the class brief before you start. This checkpoint marks the ground your team stands on before the LLM feature arrives in `PA#3`. That means the rules your assistant works under, the checks that run on every change, a CI that really refuses a bad merge, and the contract of one endpoint, with tests that hold the code to it.

Use your assistant for all of it. What we mark is what the repository shows: whether the gate bites, and whether the contract is specific enough that a stranger could build a client from it.

Due Wednesday, November 11, 2026, 22:00. One submission per team.

What is scored

#	Criterion	Points
1	Rules file	15
2	Permissions and secrets	10
3	One command for every check	10
4	CI that blocks a merge	20
5	The contract of one LeeTKOB endpoint	25
6	Contract tests that bite	15
7	`AI-LOG.md`	5

1. Rules file — 15 points

`CLAUDE.md`, `AGENTS.md`, `.cursor/rules` or `.github/copilot-instructions.md`: whichever your team's tools read (session 3, slides 4–6).

Points	What it looks like
13–15	Short, with the four sections: stack, commands, conventions, never. The *never* list is about **this** product: no secret in the sandbox, no real model call in unit tests, never edit a test, the eval data or a label to make a check pass. At least two lines can be traced to something that actually went wrong, through a commit or an `AI-LOG.md` entry.
8–12	The four sections are there, but the lines are generic ("write clean code") or none has a story.
3–7	A long file pasted from a template, or one the assistant wrote and nobody edited.
0–2	Missing.

2. Permissions and secrets — 10 points

Points	What it looks like
9–10	The assistant's permissions are written down in three tiers: allowed, ask first, never. `.env.example` lists every variable without values. No secret appears anywhere in the git history. The model API key has a spending cap, and the README says so.
5–8	`.env` is ignored and an example exists, but the permissions are implicit.
1–4	A key or password in the history, even if it was removed later. Rotate it, then say so in `AI-LOG.md` to get back to the band above.
0	A live secret in the submitted tree.

3. One command for every check — 10 points

Points	What it looks like
9–10	Lint, format check, type check (if your language has one) and tests each run with one documented command, locally and in CI, and they are **the same commands**. A fresh clone that follows the README gets them green.
5–8	The checks exist, but CI runs something different from what the README says.
1–4	Only tests, or only lint.
0	No checks.

4. CI that blocks a merge — 20 points

Points	What it looks like
17–20	`main` is protected, and the checks from criterion 3 are **required**. You link a pull request where a red check stopped the merge, and the commit that turned it green. Nothing has been pushed straight to `main` since protection was turned on.
11–16	CI runs on every pull request and goes red, but it is not required, so a red PR could have been merged.
4–10	CI exists but runs only on `main`, after the merge.
0–3	No CI, or CI that cannot fail.

5. The contract of one LeeTKOB endpoint — 25 points

In `docs/contract/<endpoint>.md`, with the same sections as `CONTRACT.md` in `IA#3`. Choose the endpoint whose errors you would least like a stranger to see. Good choices:

- `POST /labs/{labId}/submissions`: a student submits. A retried upload must not create two submissions.
- `GET /labs/{labId}/review-queue`: TA only, and only for the TA's own classes. It is a list, so it pages.
- `POST /labs/{labId}/publish`: the step that is hard to undo. It must be safe to repeat, and it must refuse while reviews are pending.
- `POST /submissions/{id}/appeals`: a student, on their own submission, only after publishing.

Points	What it looks like
21–25	**Who may call it**: the role *and* the ownership rule (whose data). The request and response shape, with field rules. Every error as problem details (RFC 9457), with each `type` listed and the status it comes with. Paging if it lists, idempotency if it creates or publishes. What happens when it changes. A stranger could write a client from this file alone.
13–20	Shape and errors are clear, but who may call it is only "authenticated", or one of paging/idempotency is missing where it applies.
5–12	An endpoint list with example bodies: no error model, no rules.
0–4	Missing, or not a LeeTKOB endpoint.

6. Contract tests that bite — 15 points

Points	What it looks like
13–15	The tests follow the contract: each error `type`, the ownership rule (another user's data gives the status your contract names), and paging or idempotency where they apply. They run in the required CI. You show one deliberate break, a commit or PR where the code violated the contract and CI went red.
8–12	The tests run in CI and cover the success path and some errors, but you never showed one failing.
3–7	Tests exist but check status codes only, or they do not run in CI.
0–2	Missing, or the tests were edited in the same commit as the code to make them pass, with no change to the contract.

7. `AI-LOG.md` — 5 points

Points	What it looks like
5	Which tool, for which part, and real *Changed* / *Rejected* entries. Example: the CI step the assistant wrote that could never fail, and how you found it.
2–4	Tools listed; *Changed* / *Rejected* are empty or generic.
0–1	Missing.

`Tool: none` is a valid entry if your team chose not to use one.

What you hand in

1. Tag the commit you are submitting `pa2` and push the tag. We check out that tag and re-run your checks.
2. The repository must be **public**. Your red team in `PA#4` reads it too.
3. Put `PA2.md` at the root, with links to: the repository, the protected-branch settings (a screenshot is fine), the blocked PR, the PR that turned green, the CI run for the tag, and the deliberate contract break.

How to submit

One zip file of the repository **including `.git`**, named after your student IDs in ascending order, joined by `-`, **and the total you give yourself**: `<StudentID1>-<StudentID2>-<StudentID3>_<total>.zip`, for example `23120001-23120002-23120003_84.zip`. One member turns it in, from their `<StudentID>@student.hcmus.edu.vn` Google account.

The zip must contain **`SELF_ASSESSMENT_REPORT.md`**: one row per criterion in this rubric, the marks you claim, and evidence pointing at a file, a commit, a PR or a CI run. Add a short *what we did not manage*. The total in that table is the number in the file name. Template on Classroom.

A wrong file name costs no marks but delays your result: the grading script matches submissions to the class list by that name.

Honesty adjustment

Your self-assessment is compared with the mark you actually earn. The gap is `your total − the mark`, on the same 100-point scale.

Gap	Adjustment
within ±10	none — this is normal calibration
+11 to +20	−3
+21 to +30	−6
more than +30	−10
−21 or worse	−3 — read the rubric before you score yourself down
no `SELF_ASSESSMENT_REPORT.md`	−10, and the file-name total is ignored

A criterion you claim with no evidence line counts as claimed-and-not-done for this comparison. The adjustment never takes a submission below 0.

How the 100 becomes your course mark

`PA#2` is 4% of the course: 100 points here are worth 4% of your course mark, and 75 points are worth 3% (0.3 on the 10-point scale).